# PLAN Y REPORTE DE MANTENIMIENTO DE SOFTWARE

**Sistema:** S.V.D.D. School — Sistema de Gestión Escolar (Python + Tkinter + SQLite)
**Versión Baseline:** v5.0 (commit 8fc2068, 03/10/2026)
**Nueva Versión:** v6.0
**Estándar de Referencia:** IEEE Std 14764-2006 / ISO/IEC 14764
**Curso:** Evolución y Mantenimiento de Software — ING-IX (2026-II)
**Responsable:** Jerson Chura
**Repositorio:** https://github.com/JersonCh1/svdd-school-mantenimiento
**Fecha:** Octubre, 2026

**Contenido**

**PLAN Y REPORTE DE MANTENIMIENTO DE SOFTWARE**


1. Introducción
2. Referencias
3. Definiciones
4. Visión General del Mantenimiento de Software
5. Proceso de Mantenimiento de Software (Iteración de 5 Cambios)
    - 5.1 Resumen del Lote de Modificaciones (MRs)
    - 5.2 Detalle de Ejecución por Fase de Mantenimiento
        - Fase A: Análisis del Problema y Modificación (Problem & Modification Analysis)
            - MR-101: Búsqueda sin tildes ni mayúsculas (Correctivo)
            - MR-102: Edición de datos del director con formulario completo (Correctivo)
            - MR-103: Base de datos en la carpeta del usuario (Adaptativo)
            - MR-104: Respaldo de la base desde la aplicación (Perfectivo)
            - MR-105: Contraseñas con hash PBKDF2 + sal (Preventivo)
        - Fase B: Implementación de las Modificaciones (Modification Implementation)
            - Código Implementado - MR-101 (Búsqueda sin tildes)
            - Código Implementado - MR-102 (Edición del director)
            - Código Implementado - MR-103, MR-104 y MR-105
        - Fase C: Pruebas del Sistema y Aceptación (System & Acceptance Testing)
            - Matriz de Casos de Prueba Ejecutados
        - Fase D: Entrega / Despliegue
6. Requisitos de Reportes de Mantenimiento
7. Requisitos Administrativos y Gestión de Configuración

Anexo

# 1. Introducción

- **1.1 Propósito:** Planificar, ejecutar y reportar una iteración de mantenimiento de cinco solicitudes de modificación (MR-101 a MR-105) sobre S.V.D.D. School, siguiendo el proceso de IEEE Std 14764-2006: análisis del problema y de la modificación, implementación, pruebas del sistema y aceptación, y entrega.
- **1.2 Alcance:** La iteración parte de la versión baseline v5.0 y produce la v6.0. Abarca la búsqueda de registros, la edición de los datos del director, la ubicación de la base de datos en el ejecutable de Windows, el respaldo de la base y el almacenamiento de contraseñas. Quedan fuera de alcance funciones nuevas como notas por matrícula o restauración de respaldos (ver Anexo).
- **1.3 Desviaciones:** Por ser un proyecto académico de un solo responsable, el CCB y el aseguramiento de calidad los asume el mismo equipo; la aprobación se respalda con 113 pruebas automáticas en lugar de una revisión independiente. MR-103 se probó simulando la carpeta de solo lectura, no con un instalador real en Windows.
- **1.4 Explicación del software:** S.V.D.D. School es un sistema de gestión escolar de escritorio escrito en Python con interfaz Tkinter y base de datos SQLite. Parte del proyecto abierto rushgala27/School-Management-System (licencia MIT) y se ha mantenido desde la v2.0. Permite iniciar sesión con tres roles (Director, Docente y Alumno). El director ve y edita sus datos y gestiona docentes, alumnos, cursos y matrículas; el docente gestiona alumnos, cursos y matrículas; el alumno consulta sus datos. Cada gestión permite registrar, consultar con búsqueda, actualizar y eliminar, con validación de datos, transacciones y claves foráneas. La base tiene cinco tablas: StudentData, TeacherData, PrincipalData, Curso y Matricula. Se distribuye también como ejecutable de Windows (.exe) generado con PyInstaller.




*Figura 1. Panel del director en la v6.0 (nuevo botón «Respaldo»).*

# 2. Referencias


- **IEEE Std 14764-2006 / ISO/IEC 14764**: *Software Engineering — Software Life Cycle Processes — Maintenance.*
- **IEEE Std 829**: Estándar para la Documentación de Pruebas de Software.
- Documento de Requisitos de Software (SRS IEEE 830) de S.V.D.D. School y registro de mantenimiento v2.0–v6.0 (docs/MANTENIMIENTO.md del repositorio).
- **Repositorio GitHub:** https://github.com/JersonCh1/svdd-school-mantenimiento (rama de esta iteración: claude/new-session-z0n3iv).
- **Proyecto original:** https://github.com/rushgala27/School-Management-System (licencia MIT).
- **Documentación técnica:** Python 3 — módulos sqlite3 (Connection.backup, create_function), hashlib.pbkdf2_hmac y unicodedata.

# 3. Definiciones

- **MR (Modification Request) / CR (Change Request):** Registro único para solicitar correcciones o mejoras.
- **CCB (Change Control Board):** Comité encargado de autorizar la ejecución de modificaciones.
- **Análisis de Impacto:** Evaluación técnica del alcance y propagación de un cambio en el código/sistema.
- **Mantenimiento correctivo / preventivo / adaptativo / perfectivo:** Clasificación según la intención (ISO/IEC 14764): reparar una falla observable / eliminar un defecto latente / adaptar el software a un cambio de entorno / añadir capacidades o mejorar usabilidad o rendimiento.
- **Tkinter:** Biblioteca estándar de Python para interfaces gráficas; el sistema usa una sola ventana con pantallas intercambiables (Frame).
- **SQLite:** Motor de base de datos embebido en un archivo (testdata.db).
- **WAL (Write-Ahead Logging):** Modo de diario de SQLite; los cambios se escriben primero en testdata.db-wal y luego se vuelcan a la base. Exige poder crear archivos junto a la base.
- **Migración idempotente:** Script que actualiza la base al iniciar y no hace nada si ya se aplicó.
- **Hash / Sal / PBKDF2:** Huella irreversible de la contraseña; la sal es un valor aleatorio por usuario; PBKDF2-HMAC-SHA256 repite el cálculo 120 000 veces para encarecer la fuerza bruta.
- **Normalización NFD / casefold:** Descomposición Unicode que separa la tilde de la letra para quitarla, y paso a minúsculas que también cubre letras no ASCII.
- **%LOCALAPPDATA%:** Carpeta de datos de cada usuario en Windows (C:\Users\<usuario>\AppData\Local).
- **PyInstaller:** Herramienta que empaqueta el programa como un único .exe para Windows.
- **Baseline:** Versión aprobada y etiquetada de la que parte la iteración (v5.0).

# 4. Visión General del Mantenimiento de Software

- **4.1 Organización:**

| Rol | Responsable | Funciones |
|---|---|---|
| **Responsable de mantenimiento** | Jerson Chura | Recibe y registra las MR, analiza el impacto, implementa y documenta. |
| **CCB** | Jerson Chura (responsable) — revisión: docente del curso | Prioriza y autoriza las MR del lote. |
| **QA / Pruebas** | Jerson Chura | Diseña y ejecuta las pruebas automáticas (unittest) y del sistema. |
| **Gestión de configuración** | Jerson Chura | Rama, commits, etiqueta v6.0 y release en GitHub. |

- **4.2 Prioridades de Programación:**

1. *P1 - Alta*: Fallas observables en la búsqueda y en la edición de datos (MR-101, MR-102).
2. *P2 - Media*: Adaptación al entorno de instalación y seguridad de contraseñas (MR-103, MR-105).
3. *P3 - Normal*: Nueva capacidad de respaldo de la base (MR-104).

- **4.3 Resumen de Recursos:** 1 persona; 1 equipo con Windows 11 (desarrollo y verificación de la interfaz) y un entorno Linux con Xvfb para las pruebas automáticas; base de datos de ejemplo testdata.db; repositorio GitHub. Esfuerzo estimado de la iteración: 12 horas (análisis 2 h, implementación 6 h, pruebas 3 h, documentación 1 h).
- **4.4 Herramientas:** Python 3.12/3.13, Tkinter, SQLite 3, Pillow, unittest, Xvfb (pruebas de interfaz sin pantalla), Git y GitHub, Visual Studio Code, PyInstaller (generación del .exe) y Claude Code como asistente de desarrollo.

# 5. Proceso de Mantenimiento de Software (Iteración de 5 Cambios)

## 5.1 Resumen del Lote de Modificaciones (MRs)

| ID MR | Nombre del Cambio | Tipo de Mantenimiento | Componente Afectado | Prioridad |
|---|---|---|---|---|
| **MR-101** | Búsqueda sin tildes ni mayúsculas | Correctivo | gestion.py (listar), db.py, texto.py (nuevo) | P1 - Alta |
| **MR-102** | Edición de datos del director con formulario completo | Correctivo | pantallas.py (formulario), gestion.py | P1 - Alta |
| **MR-103** | Base de datos en la carpeta del usuario si la del .exe es de solo lectura | Adaptativo | db.py (ruta_por_defecto, ubicar_base) | P2 - Media |
| **MR-104** | Respaldo de la base desde la aplicación | Perfectivo | db.py (respaldar), pantallas.py (panel) | P3 - Normal |
| **MR-105** | Contraseñas con hash PBKDF2 + sal | Preventivo | seguridad.py (nuevo), db.py (autenticar), migraciones.py, gestion.py | P2 - Media |

## 5.2 Detalle de Ejecución por Fase de Mantenimiento

### Fase A: Análisis del Problema y Modificación (Problem & Modification Analysis)

Para cada MR se reprodujo el problema en la baseline v5.0, se identificó la causa en el código y se evaluó el impacto. Se incluye un diagrama UML de secuencia por cambio (5 de 5) y, cuando aplica, la captura del error.

#### *MR-101: Búsqueda sin tildes ni mayúsculas (Correctivo)*

Problema: en Gestión de alumnos/docentes/cursos, buscar «lucia» no encuentra a «Lucía» y «JOSÉ» no encuentra a «José». Causa: Gestion.listar() usa LIKE de SQLite, que solo ignora mayúsculas en letras ASCII. Impacto: solo la consulta de listado (R05); no cambia datos ni esquema. Riesgo: rendimiento, porque la función plano() se ejecuta en Python por cada fila.

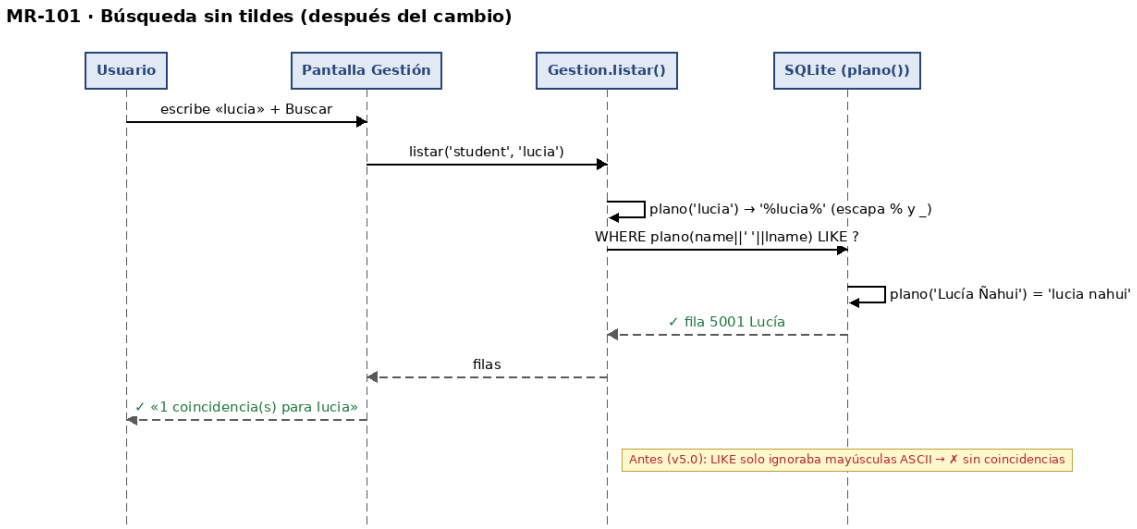



*Diagrama UML de secuencia — MR-101.*




*Antes (v5.0): «lucia» → No se encontraron coincidencias.*




*Después (v6.0): «lucia» encuentra a Lucía Ñahui.*

***MR-102: Edición de datos del director con formulario completo (Correctivo)***

Problema: «Editar mis datos» obligaba a escribir el nombre y el N.° de registro, y cambiaba un solo campo en un cuadro de texto. Fallas reproducidas en v5.0: con «Rajan » (espacio final) o «rajan» respondía «Ningún registro coincide»; y el dato se guardaba con el salto de línea del cuadro de texto ('Peruvian\n'). Impacto: pantallas.py (se elimina mostrar_edicion) y gestion.py (nuevo tipo «principal» que reutiliza validación y normalización de la v5.0).

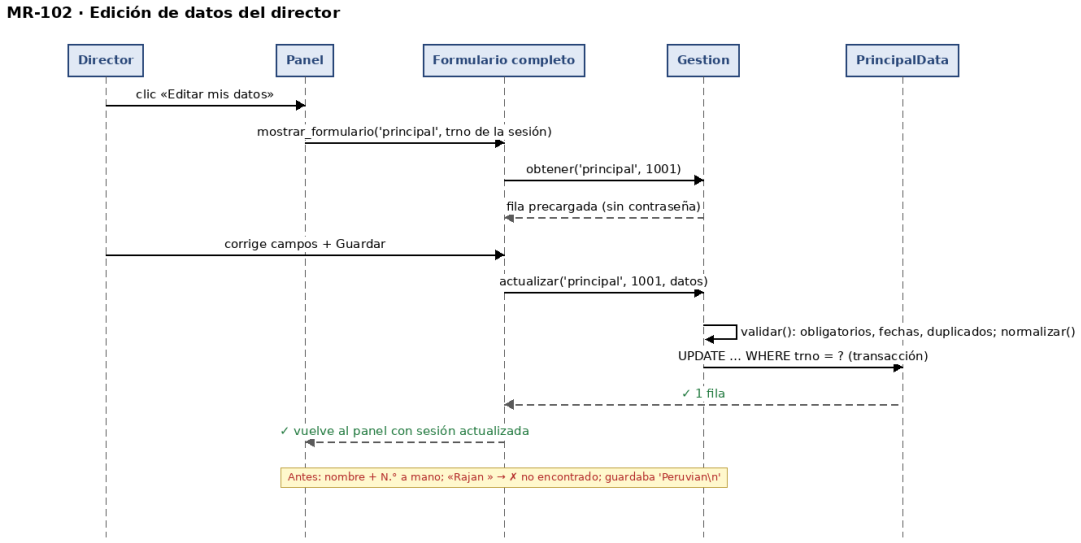



*Diagrama UML de secuencia — MR-102.*




*Antes (v5.0): edición campo por campo con datos a mano.*




*Después (v6.0): formulario completo precargado.*

***MR-103: Base de datos en la carpeta del usuario (Adaptativo)***

Problema: el .exe guarda testdata.db junto al ejecutable. Si se instala en C:\Program Files (carpeta sin permiso de escritura para el usuario), la copia inicial de la base falla con PermissionError, y el modo WAL necesita crear testdata.db-wal y -shm en esa carpeta. Impacto: solo db.ruta_por_defecto(); en modo script (python SchoolManagementSystem.py) no cambia nada.

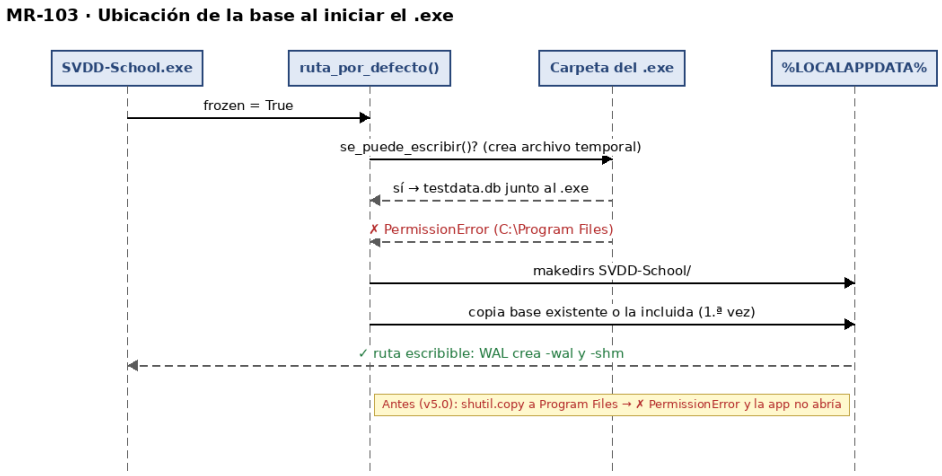



*Diagrama UML de secuencia — MR-103.*

***MR-104: Respaldo de la base desde la aplicación (Perfectivo)***

Necesidad: el colegio no tenía forma de respaldar sus datos sin cerrar la aplicación y copiar archivos a mano. Análisis: copiar testdata.db con el explorador o shutil en modo WAL puede omitir los cambios aún guardados en testdata.db-wal. Se usa la API de respaldo de SQLite. Impacto: método nuevo en db.py y un botón en el panel del director.

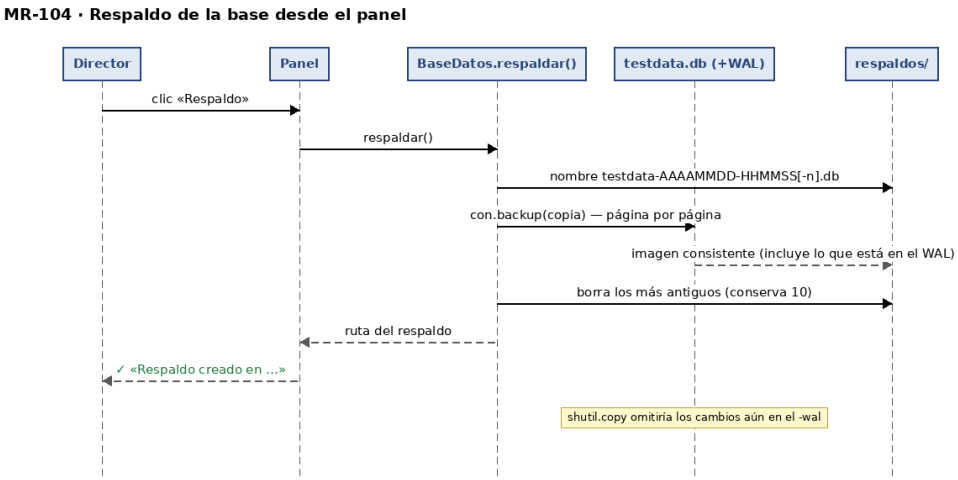



*Diagrama UML de secuencia — MR-104.*

***MR-105: Contraseñas con hash PBKDF2 + sal (Preventivo)***

Defecto latente: las contraseñas se guardan en texto plano (PrincipalData.password = 'RUpadhyay123'); cualquiera con acceso al archivo o a un respaldo (MR-104) las lee. Impacto: autenticación (db.autenticar), altas y ediciones (gestion.py), formulario (la contraseña ya no se puede mostrar) y una migración de datos para los usuarios existentes, que deben seguir entrando con la misma contraseña.

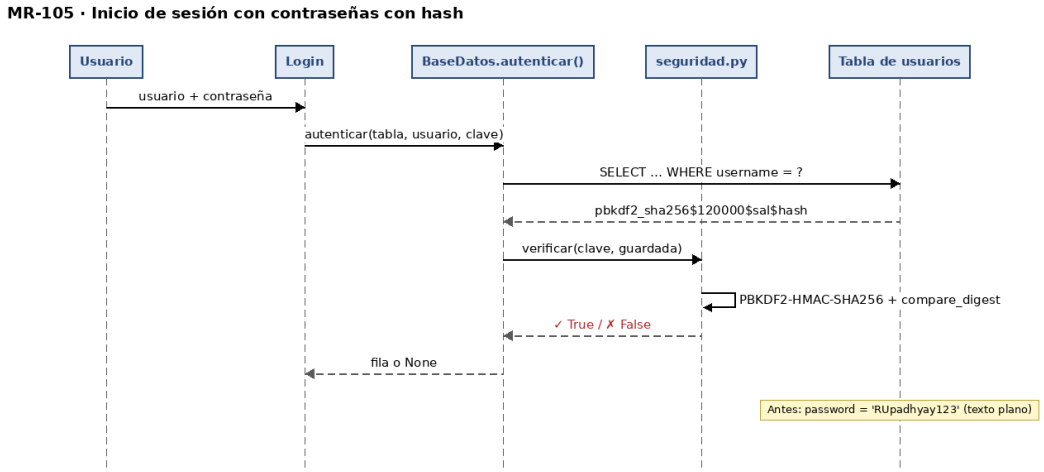



*Diagrama UML de secuencia — MR-105.*

## Fase B: Implementación de las Modificaciones (Modification Implementation)

Todas las modificaciones están en el commit 46d9781 de la rama claude/new-session-z0n3iv. Se respetó la arquitectura en capas existente: reglas en gestion.py, acceso a datos en db.py e interfaz en pantallas.py.

***Código Implementado - MR-101 (Búsqueda sin tildes)***

```
# texto.py (nuevo)
def plano(texto):
    """'Lucía Ñañez' -> 'lucia nanez'. None se queda en None."""
    if texto is None:
        return None
    descompuesto = unicodedata.normalize("NFD", str(texto))
    return "".join(c for c in descompuesto if not unicodedata.combining(c)).casefold()

# db.py — se registra la función en la conexión
self.con.create_function("plano", 1, plano, deterministic=True)

# gestion.py — Gestion.listar()
buscado = plano(filtro)
patron = "%" + buscado.replace("\\", "\\\\").replace("%", "\\%").replace("_", "\\_") +
"%"
condiciones += [f"plano({c}) LIKE ? ESCAPE '\\'" for c in busqueda["textos"]]
```




*Pantalla: «JOSÉ» encuentra a José Quispe.*

***Código Implementado - MR-102 (Edición del director)***

```
# pantallas.py — panel del director
botones += [("Editar mis datos", lambda: self.mostrar_formulario("principal",
fila["trno"])),
            ("Respaldo", self.respaldar),
            ("Docentes", lambda: self.mostrar_gestion("teacher"))]

# gestion.py — el director usa las mismas reglas que alumnos y docentes
"principal": dict(tabla="PrincipalData", clave="trno", quien="director", ...),

# pantallas.py — tras guardar, la sesión sigue abierta aunque cambie usuario o contraseña
if propio:
    rol_num, _usuario, clave_actual = self.nav.sesion
    self.nav.sesion = (rol_num, cuenta["username"].get().strip(),
                       cuenta["password"].get() or clave_actual)
volver(guardados[REGISTROS[tipo]["clave"]])
```

***Código Implementado - MR-103, MR-104 y MR-105***

*db.py — MR-103 (adaptativo)*

```
# db.py — MR-103
def se_puede_escribir(carpeta):
    # os.access() no sirve en Windows: dice que sí en Program Files.
    try:
        with tempfile.TemporaryFile(dir=carpeta):
            return True
    except OSError:
        return False

def ubicar_base(carpeta_programa, incluida):
    ruta = os.path.join(carpeta_programa, NOMBRE)
    if se_puede_escribir(carpeta_programa):
        ...
        return ruta
    datos = carpeta_de_datos()          # %LOCALAPPDATA%\SVDD-School
    os.makedirs(datos, exist_ok=True)
    destino = os.path.join(datos, NOMBRE)
    if not os.path.exists(destino):     # copia la base anterior o la incluida
        origen = ruta if os.path.exists(ruta) else incluida
        ...
```

*db.py — MR-104 (perfectivo)*

```
# db.py — MR-104
copia = sqlite3.connect(destino)
self.con.backup(copia)        # imagen consistente, incluye lo que está en el WAL
copia.close()
respaldos = sorted((f for f in os.listdir(carpeta) if ...), key=_orden_respaldo)
for viejo in respaldos[:-conservar]:
    os.remove(os.path.join(carpeta, viejo))
```

*seguridad.py y db.py — MR-105 (preventivo)*

```
# seguridad.py — MR-105
def hashear(clave, sal=None, iteraciones=ITERACIONES):
    sal = sal or secrets.token_hex(16)
    digest = hashlib.pbkdf2_hmac("sha256", clave.encode("utf-8"), bytes.fromhex(sal),
iteraciones)
    return f"{ALGORITMO}${iteraciones}${sal}${digest.hex()}"

def verificar(clave, guardada):
    if clave is None or not es_hash(guardada):
        return False                  # el texto plano nunca se acepta
    _alg, iteraciones, sal, _digest = guardada.split("$")
    return hmac.compare_digest(hashear(clave, sal, int(iteraciones)), guardada)

# db.py — la contraseña ya no se compara en el SQL
fila = self.uno(f"SELECT rowid, * FROM {tabla} WHERE username = ?", (usuario,))
if fila is None or not seguridad.verificar(clave, fila["password"]):
    return None
```

Resultado de la migración MR-105 sobre la base de ejemplo (16 usuarios):

```
Antes:  PrincipalData  RajanUp12   RUpadhyay123
Después: PrincipalData  RajanUp12   pbkdf2_sha256$120000$7cc183dadd...$165482facf6e...
```

**Fase C: Pruebas del Sistema y Aceptación (System & Acceptance Testing)**

Se ejecutó la suite completa de regresión (113 pruebas: 92 heredadas de v2.0–v5.0 y 21 nuevas en tests/test_v6.py) más pruebas del sistema de rendimiento, seguridad y recuperación. Las pruebas de interfaz abren ventanas Tkinter reales sobre un servidor gráfico virtual (Xvfb).

```
$ python -m unittest -v -b tests.test_v6
test_comodines_siguen_siendo_literales  [BusquedaSinTildesTest] ... ok
test_mayusculas_con_tilde  [BusquedaSinTildesTest] ... ok
test_plano  [BusquedaSinTildesTest] ... ok
test_sin_tilde_encuentra_con_tilde  [BusquedaSinTildesTest] ... ok
test_alta_sin_contrasena_sigue_rechazada  [ContrasenasTest] ... ok
test_alta_y_edicion  [ContrasenasTest] ... ok
test_el_hash_guardado_no_sirve_como_contrasena  [ContrasenasTest] ... ok
test_migracion_hashea_todas  [ContrasenasTest] ... ok
test_migracion_idempotente  [ContrasenasTest] ... ok
test_misma_clave_hash_distinto  [ContrasenasTest] ... ok
test_mismos_usuarios_siguen_entrando  [ContrasenasTest] ... ok
test_texto_plano_no_se_acepta  [ContrasenasTest] ... ok
test_fecha_invalida_no_se_guarda  [EdicionDirectorTest] ... ok
test_formulario_precargado_y_guardado_limpio  [EdicionDirectorTest] ... ok
test_respaldo_desde_el_panel  [EdicionDirectorTest] ... ok
test_conserva_solo_los_mas_recientes  [RespaldoTest] ... ok
test_respaldo_incluye_cambios_sin_volcar  [RespaldoTest] ... ok
test_carpeta_escribible_usa_la_del_programa  [UbicacionBaseTest] ... ok
test_conserva_la_base_que_ya_estaba_junto_al_programa  [UbicacionBaseTest] ... ok
test_se_puede_escribir_real  [UbicacionBaseTest] ... ok
test_solo_lectura_usa_carpeta_de_datos  [UbicacionBaseTest] ... ok
Ran 21 tests in 9.749s
OK

$ python -m unittest
Ran 113 tests in 46.593s
OK
```

*Figura. Reporte de ejecución de pruebas (unittest): 21/21 nuevas y 113/113 en total, OK.*

## Pruebas del sistema (no funcionales):

| Tipo | Escenario | Resultado medido |
|---|---|---|
| **Rendimiento / esfuerzo** | Búsqueda en 10 007 alumnos con nombres con tilde («lucia», «JOSÉ», «angel perez99») | v5.0: 3 ms pero 0 resultados. v6.0: 29–32 ms con 2 500, 2 500 y 29 resultados correctos. |
| **Rendimiento** | Inicio de sesión con hash PBKDF2 (120 000 iteraciones) | ≈30 ms por inicio de sesión. |
| **Seguridad** | Ataque de fuerza bruta sobre un hash robado | ≈33 intentos/s por núcleo (antes: la contraseña se leía directamente). |
| **Seguridad** | Usar el hash copiado de la base como contraseña; inyección SQL «' OR '1'='1» | Rechazado en ambos casos. |
| **Recuperación** | Respaldo con un alta aún no volcada al archivo principal (en el WAL) | La copia contiene el alta; PRAGMA integrity_check = ok; 880 KB en 5 ms. |
| **Despliegue** | Carpeta del programa sin permiso de escritura (simulada) | La base se crea en %LOCALAPPDATA%\SVDD-School y conserva los datos anteriores. |

### *Matriz de Casos de Prueba Ejecutados*

| ID Prueba | MR Relacionada | Descripción de la Prueba | Resultado Esperado | Resultado Real | Estado |
|---|---|---|---|---|---|
| **TC-101** | MR-101 | Buscar «lucia» y «JOSÉ» con alumnos Lucía | Se encuentran por nombre sin importar tildes ni | Encontrados 5001 y 5002; «%» → 0. | Aprobado |

| | | | | | |
|---|---|---|---|---|---|
| | | Ñahui y José Quispe (usuarios alu5001/alu5002) . | mayúsculas; «%» sigue sin traer a todos. | | |
| **TC-102** | MR-102 | Director pulsa «Editar mis datos», cambia el nombre a « Rajan » y guarda; luego intenta fecha 31/02/2012. | Formulario precargado, sin contraseña visible; se guarda «Rajan» sin espacios; fecha inválida rechazada. | Igual al esperado; vuelve al panel con la sesión abierta. | Aprobado |
| **TC-103** | MR-103 | Ubicar la base con la carpeta del programa sin escritura y con una base previa junto al .exe. | Ruta en %LOCALAPPDATA%\SVDD-School\ testdata.db con los datos previos. | Ruta correcta; dato «Copiado» conservado. | Aprobado |
| **TC-104** | MR-104 | Crear respaldo tras un alta no volcada; crear 5 respaldos en el mismo segundo con límite 3. | La copia incluye el alta y es íntegra; quedan los 3 más recientes. | Alta presente, integrity_check ok; 3 archivos, el último existe. | Aprobado |
| **TC-105** | MR-105 | Migrar la base, iniciar sesión con RajanUp12 / RUpadhyay123, editar alumno con contraseña vacía y luego nueva. | Todas las contraseñas con hash; login correcto; vacía conserva la anterior; la nueva reemplaza. | 16 contraseñas migradas; login ok; comportamiento esperado. | Aprobado |

## Fase D: Entrega / Despliegue

- **Versión Desplegada:** v6.0 (rama claude/new-session-z0n3iv; etiqueta v6.0 al unir a main).
- **Fecha de Despliegue:** 23/10/2026.
- **Artefactos Entregados:**

  - Código fuente v6.0: seguridad.py y texto.py (nuevos); db.py, gestion.py, migraciones.py y pantallas.py (modificados).
  - Migración de datos v6_hashear_contrasenas (idempotente, se ejecuta al iniciar).
  - Pruebas automáticas: tests/test_v6.py (21 pruebas) y ajuste de tests/test_db.py; 113 pruebas en total.
  - Diagramas UML de secuencia (docs/diagramas/v6/) y capturas antes/después (docs/capturas/v6/).
  - Registro de mantenimiento actualizado (docs/MANTENIMIENTO.md), README.md y DONDE-RETOMAR.md.
  - Este Plan y Reporte de Mantenimiento (docs/Plan_Mantenimiento_v6.docx).

# 6. Requisitos de Reportes de Mantenimiento

- **Tiempo Promedio de Resolución (MTTR):** ≈2,4 horas por MR (12 horas estimadas para 5 MR, desde el análisis hasta las pruebas en verde). Las MR correctivas (MR-101 y MR-102) fueron las más rápidas (≈1,5 h cada una) porque reutilizan componentes de la v5.0; MR-105 fue la más costosa (≈3,5 h) por la migración de datos y el ajuste de pruebas.
- **Tasa de Errores Reincidentes:** 0 % (0 de 5 MR reabiertas). La suite de regresión de 92 pruebas heredadas siguió en verde. Durante la iteración se detectaron y corrigieron antes de la entrega 2 defectos propios: el orden de borrado de respaldos (MR-104) y una prueba de búsqueda que pasaba por el usuario y no por el nombre (MR-101).

| Indicador | Valor |
|---|---|
| **MR planificadas / completadas** | 5 / 5 |
| **Pruebas automáticas (antes → después)** | 92 → 113 (100 % aprobadas) |
| **Archivos nuevos / modificados** | 2 / 4 (+ 2 de pruebas) |
| **Defectos encontrados en pruebas antes de entregar** | 2 (corregidos) |

# 7. Requisitos Administrativos y Gestión de Configuración

- **Aprobación de Calidad (QA Approval):** Las 113 pruebas de regresión y las pruebas del sistema se completaron con estado «Aprobado». La firma formal del Líder de Aseguramiento de Calidad queda a cargo de la revisión del curso.
- **Auditoría Física y Funcional (PCA/FCA):** Se verificó que el código de la rama coincide con el commit 46d9781 y los documentos con el commit siguiente de la misma rama (PCA). Cada MR tiene al menos un caso de prueba aprobado en la matriz (FCA). El .exe se regenerará con PyInstaller desde la etiqueta v6.0.
- **Gestión de configuración:** un commit por versión con mensaje «v6.0: MR-…», etiqueta anotada por versión y release en GitHub; la base de datos de ejemplo se versiona y sus cambios de esquema o datos se hacen solo con migraciones.

## ANEXO

### A.1 Fragmentos de código de las pruebas

```
class BusquedaSinTildesTest(Base):
    def test_sin_tilde_encuentra_con_tilde(self):
        self.assertEqual(self.claves("student", "lucia"), [5001])
        self.assertEqual(self.claves("student", "nahui"), [5001])

class RespaldoTest(Base):
    def test_respaldo_incluye_cambios_sin_volcar(self):
        self.g.registrar("student", {... "gno": "5001" ...})
        self.assertTrue(os.path.getsize(self.ruta + "-wal") > 0)
        destino = self.db.respaldar()
        copia = sqlite3.connect(destino)
        self.assertEqual(copia.execute("PRAGMA integrity_check").fetchone()[0], "ok")
```

### A.2 Pantallas




*v5.0: «JOSÉ» sin coincidencias.*




*v5.0: panel del director sin respaldo.*

### A.3 Modificaciones que no pudieron realizarse en esta iteración

- **Restaurar un respaldo desde la aplicación:** MR-104 solo crea copias. Restaurar exige cerrar la conexión única (Singleton), reemplazar el archivo y volver a migrar; se dejó para la v7.0 para no arriesgar la base.
- **Prueba de MR-103 en un Windows real con instalador:** las pruebas simulan la carpeta de solo lectura; falta instalar el .exe en C:\Program Files y comprobarlo con herramientas/verificar_app.py.
- **Columna normalizada con índice para la búsqueda:** plano() se evalúa por fila (≈30 ms con 10 000 alumnos). Es suficiente para un colegio; con más datos convendría una columna precalculada e indexada.
- **Fechas inválidas heredadas («tyjuyj», «jtyutr»):** no se pueden corregir automáticamente porque no se conoce la fecha real; el sistema las marca al editar el registro (v5.0 R06).
- **Selector de fecha y notas por matrícula:** requieren componentes y tablas nuevas; quedan como requerimientos perfectivos candidatos.